Predicting Follow-up Haemoglobin and Identifying Unreliable Predictions in a Tanzanian Antenatal Cohort

Technical report for the final version of the product

Josephine Duba Kanu, BSc Software Engineering, African Leadership University. Supervisor: Emmanuel Adjei. 23 October 2026

Code: github.com/DubaKanu/tanzania-hb-forecasting (branch post-feedback-revision)

Live demo: tanzania-hb-forecast-demo.onrender.com/docs

Summary. This report covers the final version of my capstone product: two machine-learning stages that forecast a woman's haemoglobin (Hb) at her next antenatal visit from ten first-visit values and estimate how far each forecast is likely to be wrong, plus a small API that shows the result. All the planned software is built and tested except the one-time locked test, which I have deliberately not run (section 1). On the 402 development women the learned reliability score lowers the error of the forecasts it keeps (0.698 g/dL at 80% coverage, against 0.794 for keeping all), but it is not clearly better than the free tree-disagreement score (difference −0.015 g/dL, 95% interval −0.058 to +0.025). The result is inconclusive and I report it that way.

1 Scope alignment

The planned scope is the one in my proposal of 18 September 2026 and my supervisor's written feedback on it: a retrospective study of the public Tanzania MHRS dataset (Mwifunyi et al., 2025), with no new data and no participants. Table 1 sets each planned feature against the final product.

Table 1. *Planned features, their purpose, status at submission and evidence*

Planned feature	Intended purpose	Current status	Evidence
Cohort audit	Rebuild the 2,685-woman cohort from 8,817 rows and count every exclusion	Done	`src/audit.py`, `reports/audit_report.md` (6,036 + 11 + 5 + 80 rows excluded)
Fixed split	1,884 training, 402 development and 399 locked-test women. Identical records stay together	Done	`split_manifest.csv`, `tests/test_pipeline.py`
Stage 1 forecaster	Forecast next-visit Hb and beat simple baselines	Done. Random forest frozen (training CV error 0.760, development 0.794 g/dL)	`reports/stage1_results.json`. Five fixed candidates, no tuning
Stage 2 error estimator	Predict each forecast's absolute error, from out-of-fold errors only	Done	`src/stage2.py`, `reports/oof_stage1.csv`, a test that in-sample residuals are never used
Selector comparison	Learned score against tree disagreement, a simple rule and random choice at 100, 90, 80 and 70% coverage, with a paired bootstrap	Done on the 402 development women. Main result inconclusive	`reports/stage2_dev_results.md`, `src/evaluation.py`
Who is set aside; complete-case check	See which women lose their forecast. Repeat everything on women with all ten inputs	Done on the development women	Same report. Of women with first Hb below 10, 54% (learned) and 46% (tree disagreement) are set aside
Protocol and locked test	Fix every choice, then score the 399 reserved women once	Protocol drafted, not frozen. **Locked test not run**	`docs/frozen_protocol.md`, `src/run_locked_test.py`, `tests/test_locked_test_guards.py`
Researcher demo API	Ten first-visit values in, forecast, estimated error and kept or set-aside flag out. Eight acceptance checks	Done. 8 of 8 pass. Live on Render	`app/`, `tests/test_api.py`, `docs/acceptance_checks.md`
Data documentation	Data dictionary, exclusion log, provenance, README	Done	`docs/data_dictionary.md`, `docs/data_provenance.md`, `README.md`
Proposal revision	Revised text, remaining figures and literature search log (response to the feedback)	Open. Not part of the software. Figure 4 is redrawn from the code	`docs/response_table.md`

Scope changes and why.

- **Locked test not run.** The 399 women can be used once. It waits until my supervisor has reviewed the protocol, including the items I marked as proposed, and tells me I may run it. Running it now could spend the only unbiased test on a protocol he may still change. The code refuses to run it twice, or after any frozen file has changed.
- **No tuning.** After his comment on method, every setting is fixed in advance (five candidates, no parameter search), so a held-out row cannot influence the forecasts whose errors Stage 2 learns from.
- **Simple-rule baseline added** (proposed, not yet approved), because his test of the contribution is that a simple rule must not be able to replace the model. It ranks by the number of missing inputs and breaks ties by the distance of first Hb from the training median.
- **A cutoff for the API.** One woman cannot be ranked, so the demo flags a forecast with a fixed cutoff (the 80th percentile of development scores, 0.915 g/dL). It is labelled as a different method from the exact-coverage rule of the evaluation.
- **Data handling.** The raw clinical file is no longer in the current version of the repository. Earlier commits still hold it, and what to do about that is my supervisor's decision. The loader reads 12 of the 683 columns and never the dataset's own risk label.

- **Added in this version:** the class design, hardened API inputs, a faster forecast path, a reproducibility check and a CI workflow. They answer the testing and code-quality expectations of this assignment and change no result (Table 4).

2 Technical functionality

Data. The raw file has 8,817 rows and 683 columns. `data_adapter.py` reads 12 of them, turns "not_checked" into missing values and splits blood pressure into systolic and diastolic. A woman is eligible with two valid, correctly ordered Hb readings, which leaves 2,685. A seeded permutation (seed 42) makes the three partitions.

Stage 1. Five candidates with fixed settings: the training median, carrying the first Hb forward, a line on first Hb and gestation, ridge regression and a random forest (200 trees, depth 6, minimum leaf 5; Breiman, 2001; Pedregosa et al., 2011). Imputation and scaling are fitted inside each fold of a 3-fold cross-validation that keeps identical records together. The forest has the lowest error (0.760 against 0.800 for ridge) and is frozen.

Stage 2. A second, separate random forest learns the absolute error of each Stage 1 forecast. Its training table has one row per training woman and 12 inputs: the ten first-visit values, the out-of-fold forecast and the tree disagreement (the standard deviation of the 200 trees' forecasts). Every forecast in the table comes from a fold model that never saw that woman, so the errors are honest. For a new woman the frozen forest gives the forecast and the disagreement, and Stage 2 turns them into an estimated error.

Choosing which forecasts to keep. Each selector scores every woman and the lowest scores are kept: at coverage c the n = round(c x N) lowest (319 of 399 at 80%), ties broken by record ID. All selectors use the same Stage 1 forecasts, and the bootstrap resamples whole groups of identical records.

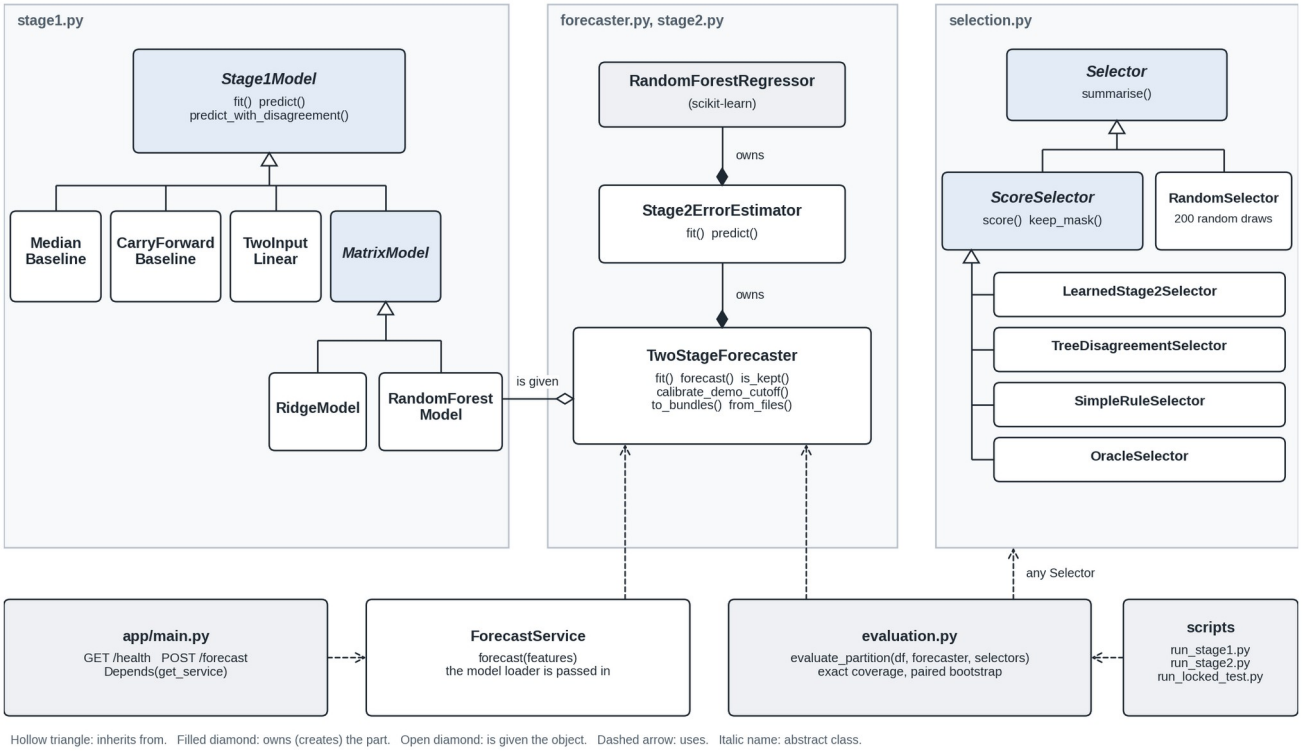


Figure 1. *The classes of the code, drawn from the code by `scripts/make_class_diagram.py`. A hollow triangle means inherits from, a filled diamond owns, an open diamond is given, a dashed arrow uses.*

Table 2. *The main algorithms and why each was chosen*

Algorithm	Problem it solves	How it works and what it costs
Fold rule (`assign_cv_folds`)	Folds must be the same on every library version and must never split identical records	Groups largest first, each to the fold with the fewest rows so far, ties to the lowest fold number. One sort and one pass, O(G log G) for G groups. This greedy rule keeps the largest fold within 4/3 of the best possible (Graham, 1969). It replaced `GroupKFold`, whose tie-breaking changed between versions and moved the CV error by about 0.003 g/dL.
Exact-coverage selection (`keep_mask`)	Compare selectors on exactly the same number of kept forecasts, whatever a sort library does with ties	One `lexsort` on (score, record ID), then keep the first n = round half up of c x N. A guard handles floating-point noise (0.7 x 45 is 31.499999999999996 and must keep 32). O(N log N).
Tree disagreement in one pass (`tree_predictions`, `forest_mean`)	scikit-learn re-checked its input for each of the 200 trees, three times per forecast	Convert the input to float32 once and ask each fitted tree directly. Add the trees one at a time in scikit-learn's own order, so the mean equals `predict` to the last digit. The same per-tree values give the standard deviation. One forecast fell from 73.7 to 7.3 ms.

Algorithm	Problem it solves	How it works and what it costs
Paired group bootstrap (`paired_bootstrap_selectors`)	Say how sure we are of a difference between two selectors (Efron & Tibshirani, 1993)	5,000 replicates. Each resamples the groups with replacement, re-applies the exact-coverage rule for every selector on the same resampled women and records the differences. Percentile 95% interval.
Out-of-fold table (`build_oof_table`)	Stage 2 must learn from errors the forest did not make on its own training rows	Run the 3-fold loop with the fixed recipe, keep each woman's forecast and disagreement from the fold model that never saw her, then fit Stage 2 once on the whole table.

The API. `POST /forecast` takes the ten inputs (six required, four optional) and returns the forecast, the estimated error, the flag, the cutoff, a plain-language note and a disclaimer. `GET /health` reports whether the models are loaded. There is no separate front end: the interface is the Swagger page that FastAPI builds from the same schemas that check the input, so the page and the back end cannot disagree about the inputs. One request goes through six steps. (1) Pydantic checks every range before any model code runs, refuses NaN and Infinity, and a 422 message never repeats the value sent. (2) FastAPI hands the route the shared `ForecastService`, which is given a loader function, not a file path. (3) On the first valid request it reads the two model files, under a lock, or answers 503 with instructions if they are missing. (4) `TwoStageForecaster.forecast` fills missing inputs with the training medians, takes the forecast and the tree disagreement from the Stage 1 forest, builds the 12 Stage 2 inputs and estimates the error. (5) The flag is *kept* if the estimate is at or below the cutoff. (6) `ForecastResponse.from_verdict` rounds the numbers, writes the note and adds the disclaimer. The evaluation and the locked test use the same `TwoStageForecaster`, so the API returns what the offline code returns. Nothing is stored. The demo runs on Render's free plan.

Table 3. *Mean absolute error (g/dL) of the forecasts kept, 402 development women; lower is better*

Selector	100%	90%	80%	70%
Learned Stage 2 score	0.794	0.753	0.698	0.693
Tree disagreement	0.794	0.764	0.713	0.671
Simple rule	0.794	0.759	0.796	0.771
Random selection (mean of 200)	0.794	0.794	0.794	0.793
Oracle (needs the answer; reference only)	0.794	0.598	0.482	0.394

What the results show. Both scores beat random choice. The learned score minus tree disagreement at 80% is −0.015 g/dL (95% interval −0.058 to +0.025), so the study cannot say which is better and does not show that they are equivalent. The simple rule is no better than keeping everything. The extra inputs add little: the forest beats a two-input line by 0.050 g/dL on training out-of-fold errors (interval −0.064 to −0.036) but only by 0.017 on the development women (interval −0.045 to +0.011).

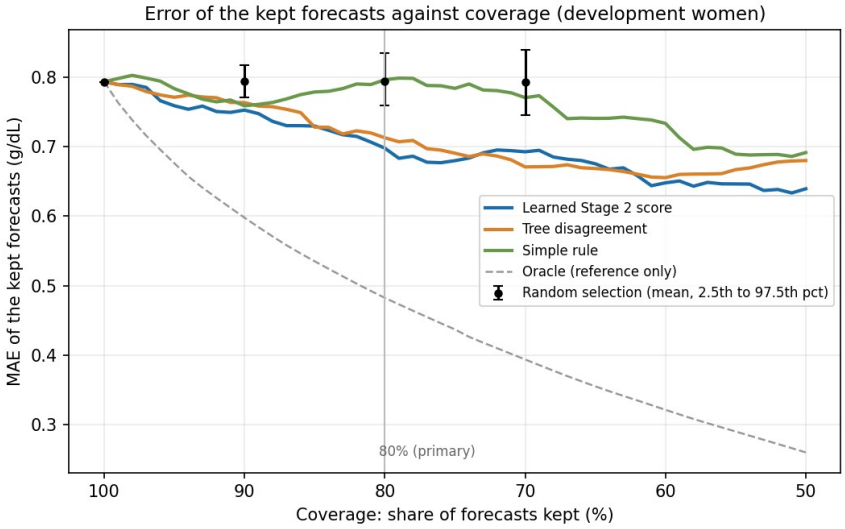


Figure 2. *Error of the forecasts kept, against the share kept (`reports/fig_error_vs_coverage_dev.png`). The learned score and tree disagreement run close together and both stay below random selection.*

3 Testing approaches and results

The suite has 316 tests in 16 files (pytest), plus checks outside pytest. Table 4 gives, for each strategy, why I chose it, how I ran it, and what it found. The counts in its first column add up to the 316.

Table 4. *Testing strategies, results and the improvements they led to*

Strategy	Why chosen	How conducted	Results and improvements
1. Unit and integration tests of the modelling code (101 tests, plus 11 locked-test guards)	A leak or an off-by-one in the coverage rule would silently change every result	Small synthetic cohorts with known answers. Reference versions written by hand (the bootstrap is compared with a plain loop). Checks that each out-of-fold forecast comes from a model that never saw that woman, that the five Stage 1 models and the selectors obey one interface, and that the locked test refuses to run twice	All pass on Python 3.9, 3.10, 3.11 and 3.12. They were my safety net for moving the code into classes (row 4). One test pins the guard in `n_keep` (0.7 x 45 is 31.499999999999996 and must keep 32). A test of the order of steps in the locked-test script failed when I added docstrings, and now looks past them
2. API input and boundary testing (148 tests)	The API is the part other people use, with values I did not expect	Each input at its lowest and highest allowed value and 0.01 beyond; NaN, Infinity, 1e39, 1e300; wrong types; null; all 16 patterns of left-out optional inputs; broken bodies; extra fields; wrong methods; missing model files	It failed at first: NaN, Infinity or a huge number in eight inputs gave HTTP 500 and not 422 (the first submitted version did the same). Fixed with limits on every input, `allow_inf_nan=False` and a handler that does not echo the value. All cases are kept as regression tests
3. Acceptance and API-layer tests (8 agreed checks; 42 tests)	These define a working demo, as agreed with my supervisor	Seven automated checks in `tests/test_api.py` (22 tests), the service without the web layer (14) and missing model files (6); one fresh-environment build from the README. The live copy was checked by hand on 18 October	8 of 8 pass. The live copy returned the same numbers as the local one
4. Reproducibility testing (11 tests and a script)	Every result must come back from the raw file after I reorganised the code	`scripts/check_reproducibility.py` copies the repository, deletes the 19 derived files, rebuilds them with the three scripts and compares bytes	Python 3.9.25, which built the committed files: 19 of 19 identical. Python 3.10, 3.11, 3.12: 14 identical, 5 identical except the saved Python version. No number changed. An earlier comparison of environments had shown that `GroupKFold` moved the error by 0.003 g/dL, which is why the code has its own fold rule
5. Performance testing (3 tests and a script)	The demo runs on a small free server	`scripts/benchmark_api.py`: 300 requests one at a time after 10 warm-ups, then 1, 4 and 16 clients at once, memory from `ps`, and a profile of one forecast	Table 5. The profile found the repeated input checks; calling the trees directly gave identical numbers (`array_equal` on 2,286 women)
6. Statistical validation of the models	A forecast must hold on women the model never saw, and a difference only counts if it is larger than the noise	Group-aware 3-fold cross-validation inside the training women, a separate development partition for every comparison, paired bootstrap intervals, a repeat on complete-case women, and the locked test held back	Stage 1 error 0.760 in cross-validation and 0.794 on the development women. The intervals separate real differences (forest against a two-input line is clear in cross-validation, not on development women) from inconclusive ones (learned against tree disagreement)

Table 5. *Performance of the API on 2 vCPU (Intel Xeon 2.80 GHz), 8 GB, Ubuntu 24.04, Python 3.11.17, before and after the fix*

Measure	Before	After	Measure	Before	After
Median request, one client	73.5 ms	11.1 ms	Requests per second, 1 client	12.6	68.8
95th percentile	107.4 ms	17.2 ms	Requests per second, 4 clients	8.2	38.7
First request (loads models)	192 ms	82 ms	Median wait, 16 clients	2,014 ms	480 ms
Memory after the load test	183 MB	182 MB	402 women in one call	0.118 s	0.022 s

With the server pinned to one core the median is 10.5 ms at about 80 requests per second. Table 6 shows the software environments. The hosted copy runs on Render's free plan (Python 3.11.9, 512 MB), where the first request after 15 minutes of silence waits for the service to wake up.

Table 6. *Compatibility with the pinned libraries (`requirements.txt`)*

Environment	Install	Tests	Rebuild of the 19 derived files from the raw file
Python 3.9.25, Linux	ok	316 pass	19 identical, byte for byte
Python 3.10.20, 3.11.17 and 3.12.13, Linux	ok	316 pass on each	14 identical, 5 identical except the saved Python version
Python 3.13.17, Linux	fails: `scipy` 1.13.1 has no ready-made build and needs a Fortran compiler	not run	not run
Render free plan, Python 3.11.9	ok, live	4 hand checks, 18 October	not applicable

Not tested: macOS, Windows, other hardware, and the speed of the hosted copy.

4 Code quality practices

Table 7. *Object-oriented design and the SOLID principles (Martin, 2003) in the code*

Principle	How the code applies it	Evidence
Single responsibility	One job per module: `metrics.py` (error and coverage maths), `selection.py`, `evaluation.py` (comparison tables), `reporting.py` (text and figures), `provenance.py`. In `app/`: `schemas.py` (request and response), `service.py` (the forecasting job), `errors.py`, `main.py` (routes only). Scripts are short lists of steps and nothing runs on import	15 modules in `src/`, the largest 325 lines. 155 functions, mean 9.9 lines, longest 32
Open / closed	A new Stage 1 model or a new selector is one new subclass. The cross-validation loop and `evaluate_partition` do not change	`Stage1Model`: five models. `Selector`: five selectors
Liskov substitution	Any subclass works where its base class is expected, and the tests run the same checks over all of them	`tests/test_stage1_models.py`, `tests/test_selection.py`
Interface segregation	A selector needs only `summarise`. A `ScoreSelector` adds `score`. `RandomSelector` has no score, so it is not one	`src/selection.py`
Dependency inversion	`ForecastService` is given a loader function and FastAPI injects it with `Depends`. `TwoStageForecaster` is given its fitted models. Tests pass stubs	`tests/test_api_errors.py`, `tests/test_service.py`
Composition	`Stage2ErrorEstimator` owns its forest and `TwoStageForecaster` owns the estimator, as in the proposal's class design	Figure 1

Other practices.

- **Readability.** Every module, class and function in `src/` and `app/` has a docstring. Type hints, descriptive names and settings kept in one place (`STAGE1_CONFIG`, the protocol constants).
- **Static checks.** `ruff check .` (pycodestyle, pyflakes, bugbear and a complexity limit of 10) passes with no warnings. Radon rates the 243 classes, functions and methods in `src/`, `app/` and `scripts/` at a mean cyclomatic complexity of 2.2 (McCabe, 1976), 15 at grade B and none worse, and all 20 modules of `src/` and `app/` at maintainability grade A.
- **Reproducibility.** Seeds are fixed, libraries are pinned, the result reports record their run ID, library versions and file checksums, and the saved models are plain scikit-learn objects, so they load without this code's classes.
- **Version control.** The first submission is kept as the release tag `initial-demo-submitted`. A GitHub Actions workflow repeats the four-version tests and the style check on every upload.

5 Limitations and next steps

- The results are on the 402 development women only, and the main comparison is inconclusive. The locked test follows my supervisor's review of the protocol.
- Validation is internal. There is no verified patient identifier, so grouping identical records is only a proxy for the same woman. The data come from five Tanzanian districts, the forecast horizon varies from 1 to 28 weeks (median 4), and clinical usefulness is untested. The demo is for made-up values only.
- Speed and compatibility were measured on one Linux machine. The next steps are to measure the hosted copy, freeze the protocol, run the locked test once and finish the proposal revision.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics, 17*(2), 416-429. https://doi.org/10.1137/0117039

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308-320. https://doi.org/10.1109/TSE.1976.233837

Mwifunyi, R., Kibinda, N., Elisa, N., Rashidi, F. U., Hassan, K., Mohsini, M., Masoi, T., Mnabwiru, L., Hussein, Y., & Manangwa, S. (2025). *Dataset for Maternal Health Risks Stratification (MHRS) in Tanzania* (Version 1) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.15309733

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.